TEMA 2: Sintaxis del Lenguaje JavaScript

2.1. Selección de un lenguaje de programación de clientes Web (Criterio 2.1)

- **Resultado de Aprendizaje 2 (RA2):** *Escribe sentencias simples, aplicando la sintaxis del lenguaje y verificando su ejecución sobre navegadores Web.*

- **Criterio Curricular Oficial: CE 2.a** (Se ha seleccionado un lenguaje de programación de clientes web en función de sus posibilidades.).

- **Ponderación del Criterio:** (**10% del RA2** | **0,5% sobre la calificación final del módulo**).

La elección de una tecnología para el desarrollo en el entorno cliente determina la compatibilidad, el rendimiento, la mantenibilidad y la experiencia de usuario de cualquier aplicación web. Aunque históricamente han existido diversas alternativas para dotar de dinamismo al navegador, **JavaScript (ECMAScript)** se ha consolidado como el estándar universal indiscutible respaldado por la industria y los organismos internacionales.

EVOLUCIÓN Y SELECCIÓN DE TECNOLOGÍAS EN CLIENTE

1. La Era Oscura: Alternativas Históricas (Descartadas)

A finales de los 90 y principios de los 2000, los navegadores eran muy limitados. Para crear páginas interactivas o juegos, se dependía de complementos (*plugins*) externos:

- **Java Applets:** Intentaron llevar la potencia de Java al navegador, pero eran pesados, tardaban mucho en cargar y se convirtieron en un coladero de agujeros de seguridad.
- **Adobe Flash:** Fue el rey de la multimedia y los juegos web. Sin embargo, su destino quedó sellado cuando Steve Jobs decidió no soportarlo en el iPhone por su altísimo consumo de batería y falta de optimización.
- **VBScript:** El intento de Microsoft de competir con JavaScript. Al funcionar solo en Internet Explorer, fragmentó la web y fracasó en la búsqueda de un estándar.

2. La Consolidación: Estándar Universal

Hoy en día, el **100% de los navegadores modernos** (Chrome, Safari, Edge, Firefox) entienden un único lenguaje nativo: **JavaScript (bajo el estándar ECMAScript)**.

Cualquier cosa que se ejecute en el cliente debe pasar por el motor de JavaScript del navegador.

3. La Modernidad: Superoferta y Dialectos

Como JavaScript original (*Vanilla JS*) podía volverse caótico en proyectos gigantescos, la industria creó herramientas que mejoran la experiencia de desarrollo, pero con una

condición: **al final, todo se traduce (compila/transpila) a JavaScript puro** para que el navegador lo entienda.

- **TypeScript:** Añade "tipado estático" a JavaScript. Es decir, te obliga a definir si una variable es un texto, un número o una fecha. Esto evita que cometas errores en el código antes de que la página se suba a producción.
- **JSX:** Es una sintaxis que permite mezclar código HTML directamente dentro de JavaScript. Es la pieza clave de librerías como **React** para crear componentes visuales reutilizables.

A. Razones Técnicas para la Selección de JavaScript en Front-end

Al evaluar un lenguaje para programar en el cliente frente a especificaciones técnicas y comerciales, JavaScript se selecciona por sus ventajas arquitectónicas:

- **1. Estandarización e interoperabilidad sin extensiones (*Zero Plugins*):**
  - Tecnologías históricas requerían instalar complementos externos pesados en el navegador del usuario (como el entorno de ejecución de Java para los *Applets* o el reproductor de Adobe Flash).
  - JavaScript es interpretado de forma nativa por todos los motores de renderizado modernos (V8 en Chromium/Edge, SpiderMonkey en Firefox y JavaScriptCore en Safari) sin exigir instalaciones adicionales al usuario.
- **2. Reducción de la carga de cálculo y tráfico en el servidor:**
  - Al delegar la ejecución de la lógica inmediata (validaciones, cálculos de interfaz, ordenaciones visuales) en la máquina del cliente, el servidor reduce su consumo de CPU y memoria.
  - Se transfieren únicamente datos puros (generalmente en JSON) en lugar de documentos HTML completos, optimizando el ancho de banda.
- **3. Modelo asíncrono y no bloqueante (Orientado a Eventos):**
  - La arquitectura de JavaScript se basa en un bucle de eventos (*Event Loop*). Permite atender clics, animaciones o peticiones de datos de fondo sin congelar la ventana del navegador.
- **4. Ecosistema universal (*Full-stack*):**
  - Permite unificar el lenguaje en ambos lados de la arquitectura: JavaScript en el navegador (front-end) y JavaScript en el servidor mediante entornos como Node.js (back-end), facilitando la reutilización de modelos y funciones comunes.

B. Reglas sintácticas universales del lenguaje

- **1. El punto y coma (;):**
  - El punto y coma se utiliza para separar sentencias en JavaScript.
  - *Regla técnica:* Aunque en muchas situaciones no es obligatorio si se introduce un salto de línea (gracias al mecanismo de inserción automática de punto y coma o *ASI - Automatic Semicolon Insertion*), **es una buena**

práctica y altamente aconsejable utilizarlo siempre para evitar ambigüedades e interpretaciones erróneas del motor.

- **2. Sensibilidad a mayúsculas y minúsculas (*Case-sensitive*):**
  - JavaScript distingue estrictamente mayúsculas de minúsculas no solo en identificadores inventados por el usuario, sino en palabras reservadas y funciones nativas.
  - *Ejemplo:* Si se escribe Time en vez de time, o Alert() en vez de alert(), el script producirá un error sintáctico o de referencia en tiempo de ejecución.
- **3. Nombres de identificadores (variables, funciones, objetos):**
  - *Reglas obligatorias de nomenclatura:*
    - Deben comenzar obligatoriamente por una letra (a-z, A-Z), el signo de dólar ($) o un guion bajo (_).
    - **Nunca pueden comenzar por un número o dígito** (un identificador como 1variable es sintácticamente inválido).
    - No pueden utilizarse palabras reservadas del lenguaje (break, case, catch, class, const, continue, debugger, default, delete, do, else, export, extends, finally, for, function, if, import, in, instanceof, new, return, super, switch, this, throw, try, typeof, var, void, while, with, yield).
- **4. Conjunto de caracteres y codificación:**
  - JavaScript utiliza el juego de caracteres **Unicode**, lo que permite emplear prácticamente cualquier carácter o símbolo internacional en cadenas y textos literales.

El conflicto del punto y coma: ASI (*Automatic Semicolon Insertion*)

Se insiste en usar siempre el punto y coma (;), pero en el desarrollo profesional existe un debate habitual (estilos con punto y coma frente a estilos limpios tipo *StandardJS*). Conviene explicar **por qué** recomendamos ponerlo siempre:

- JavaScript dispone de un mecanismo interno llamado **ASI (*Automatic Semicolon Insertion*)** mediante el cual el motor intenta adivinar dónde termina una sentencia si hay un salto de línea.
- **El peligro real del ASI (Caso de retorno vacío):**

```
// Lo que el programador escribe:
function obtenerUsuario() {
  return
  {
    nombre: "Ana"
  };
}
// Lo que el motor interpreta por culpa del ASI:
function obtenerUsuario() {
```

```
  return; // <-- Inserta un punto y coma automático aquí
  {
    nombre: "Ana"
  };
}
// Resultado: la función devuelve 'undefined' en vez del objeto.
```

Convenciones de Nomenclatura Profesional (Clean Code)

Además de las restricciones sintácticas que prohíben empezar con dígitos o usar palabras reservadas, en el entorno laboral se aplican convenciones universales que el alumnado debe adquirir desde el primer tema:

- **camelCase (o joroba de camello):** Estándar para variables, propiedades y funciones (totalFactura, calcularImporte(), nombreUsuario).
- **PascalCase (o UpperCamelCase):** Reservado para nombres de clases, componentes (React) y funciones constructoras (UsuarioRegistrado, FacturaService).
- **UPPER_SNAKE_CASE:** Constantes globales inmutables cuyos valores se conocen en tiempo de diseño (MAX_REINTENTOS, PI, URL_API).
- **Prefijo $:** Común en librerías para referenciar elementos directos del DOM (const $botonEnviar = ...).
- **Prefijo _:** Convención para denotar propiedades o métodos con intención privada dentro de objetos o clases.

C. Especificación del Lenguaje: De JavaScript a ECMAScript (ES6 / ES2015)

- **¿Qué es ECMAScript?**
  - Es la especificación formal estandarizada por la organización internacional **ECMA International** (especificación ECMA-262) que define la sintaxis, tipos y comportamientos que deben cumplir los motores de los navegadores.
  - JavaScript es la implementación real de dicha norma técnica.
- **El salto de ES6 (ES2015):**
  - Marcó la transición hacia el desarrollo profesional moderno mediante la introducción de características como:
    - Declaración de variables con ámbito de bloque (let y const), corrigiendo las fugas de ámbito de var.
    - Sintaxis formal de clases (class), facilitando la orientación a objetos.
    - Funciones flecha (=>) para una sintaxis más limpia y mejor manejo del contexto this.
    - Métodos declarativos avanzados para matrices (map, filter, reduce).

D. Evaluación de Alternativas: JavaScript frente a TypeScript y JSX

En función de los requisitos del proyecto, el desarrollador selecciona si codifica en JavaScript puro o utiliza dialectos que compilan (*transpilan*) a JavaScript:

Criterio de Selección	**JavaScript Nativo (Vanilla JS)**	**TypeScript**	**JSX (JavaScript XML)**
Definición	Lenguaje estándar nativo interpretado por navegadores.	Superconjunto tipado de código abierto creado por Microsoft.	Extensión de sintaxis para React parecida a plantillas con JS integrado.
Tipado de datos	Débil y dinámico (las variables cambian de tipo en ejecución).	Fuerte y estático (se declaran tipos para variables, parámetros y retornos).	Hereda el tipado de JavaScript subyacente.
Fase de compilación	Ninguna; se descarga y ejecuta directamente en el navegador.	Obligatoria; se compila a JavaScript estándar para poder ejecutarse.	Obligatoria; herramientas como Babel la transforman en llamadas JS convencionales.
Detección de errores	En tiempo de ejecución (*runtime*).	En tiempo de desarrollo y compilación previa.	Al compilar el árbol de componentes.
Proyecto recomendado	Scripts de tamaño pequeño a medio, manipulación directa del DOM.	Aplicaciones corporativas de gran escala (estándar en Angular).	Aplicaciones orientadas a componentes en ReactJS.

E. Glosario Técnico Asociado a la Selección del Lenguaje

- **Case-sensitive:** Expresión técnica que indica que el lenguaje es "sensible a mayúsculas y minúsculas". En JavaScript, identificadores como miVariable y Mivariable representan dos posiciones de memoria totalmente independientes.
- **Consola de JavaScript:** Panel de diagnóstico técnico accesible comúnmente con la tecla **F12** que permite comunicar el script con el exterior mediante console.log() para verificar variables y depurar fallos.
- **Transpilador:** Compilador específico que traduce código fuente escrito en un lenguaje o dialecto moderno (como TypeScript o ES2020) a código fuente JavaScript estándar compatible con navegadores más antiguos (ejemplo: Babel o el compilador de TypeScript tsc).

F. Actividades Prácticas y Ejercicios del Criterio 2.1

1. **Análisis de Tecnologías Front-end:**
   Clasifica las siguientes opciones en *Alternativas Históricas Obsoletas*, *Estándar Nativo Universal* o *Dialectos/Superconjuntos con Transpilación*:
   - Adobe Flash (ActionScript)
   - TypeScript
   - Java Applets
   - JavaScript (ECMAScript)
   - JSX
   - VBScript
2. **Justificación de Arquitectura:**
   Un equipo de desarrollo está debatiendo la arquitectura para una nueva aplicación web financiera.
   - Explica dos ventajas técnicas de utilizar JavaScript nativo en el cliente para reducir el uso de recursos en el servidor.
   - Define el concepto de *Zero Plugins* y menciona qué problema histórico soluciona frente a tecnologías antiguas.
3. **Matriz de Selección de Dialectos:**
   A partir de los requisitos de tres proyectos distintos, indica la tecnología recomendada (*Vanilla JS*, *TypeScript* o *JSX*) y justifica tu respuesta en base al tipo de proyecto y la fase de detección de errores:
   - **Proyecto A:** Una plataforma empresarial bancaria con un equipo de 30 desarrolladores que requiere validación estática de datos en tiempo de desarrollo.
   - **Proyecto B:** Un script ligero de 50 líneas para manipular el DOM e integrar un widget flotante en un sitio web estático.
   - **Proyecto C:** Una interfaz de usuario interactiva basada en componentes reutilizables utilizando ReactJS.
4. **ECMAScript y Transpilación:**
   Define la relación entre **ECMAScript** y **JavaScript**. Explica qué papel desempeña un **transpilador** (como Babel) cuando se utiliza sintaxis moderna de ES6/ES2015 en entornos corporativos que deben mantener compatibilidad con navegadores antiguos.
5. **Validación Sintáctica de Identificadores:**
   Analiza la siguiente lista de nombres de variables. Indica cuáles son **válidos** y

cuáles son **inválidos** según las reglas de JavaScript. Para los inválidos, especifica el motivo exacto de la infracción:

- 1erUsuario
- _totalFactura
- $elementoDOM
- precio-final
- class
- montoTotal2
- nombre usuario
- function

6. **Detección de Errores por Sensibilidad a Mayúsculas (*Case-sensitive*):**
   El siguiente código JavaScript presenta múltiples errores sintácticos o de referencia debido al mal uso de mayúsculas y minúsculas. Identifica los 4 errores presentes y reescribe el fragmento corregido:

```
Function calcularDescuento(PrecioBase) {
  Const porcentaje = 0.15;
  If (PrecioBase > 100) {
    Alert("Descuento aplicado");
    Return PrecioBase * porcentaje;
  }
  return 0;
}

Console.log(calcularDescuento(150));
```

7. **Aplicación de Convenciones Profesionales:** Refactoriza los nombres de las siguientes entidades aplicando la convención de nomenclatura adecuada (*camelCase*, *PascalCase*, *UPPER_SNAKE_CASE*, *$* o *_*):
   - Una variable para almacenar el nombre del cliente actual.
   - Una constante global inmutable para el número máximo de reintentos de conexión.
   - Una clase que representa a un usuario registrado en el sistema.
   - Una variable que almacena una referencia directa a un botón del DOM.
   - Una propiedad de un objeto con intención de uso privado para guardar el token de sesión.

8. **Análisis del Comportamiento del ASI (Inserción Automática de Punto y Coma):**
   Observa el siguiente bloque de código:

```
function crearConfiguracion() {
  return
  {
    modo: "oscuro",
    puerto: 8080
```

```
  };
}

const config = crearConfiguracion();
console.log(config);
```

- **a)** ¿Qué valor se imprimirá exactamente en la consola del navegador al ejecutar este script?
- **b)** Explica paso a paso cómo interviene el mecanismo ASI del motor de JavaScript para producir dicho resultado.
- **c)** Reescribe el código corregido aplicando las buenas prácticas marcadas en el temario.

9. **Verificación en Consola y Ámbito:**
   Indica qué salida se obtiene en la consola al ejecutar de forma secuencial las siguientes líneas y explica el comportamiento del motor respecto a identificadores independientes:

```
let totalVentas = 500;
let TotalVentas = 1200;

console.log(totalVentas);
console.log(TotalVentas);
```